AFRILANG Stdlib

std/c/parsehtml

Français. Bibliothèque AFRILANG ‘std/c/parsehtml‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : htmlLen, htmlUpper, htmlLower, htmlTrim, htmlSplit, htmlJoin, htmlReplace.
‘import ”std/c/parsehtml”‘ · ‘use parsehtml‘
- ‘htmlLen(s text) → number‘ - ‘htmlUpper(s text) → text‘ - ‘htmlLower(s text) → text‘ - ‘htmlTrim(s text) → text‘ - ‘htmlSplit(s text, delim text) → list text‘ - ‘htmlJoin(parts list text, delim text) → text‘ - ‘htmlReplace(s text, from text, to text) → text‘ **English.** AFRILANG library ‘std/c/parse-

html‘ (tier complex, category complex). Exports 7 function(s): htmlLen, htmlUpper, htmlLower, htmlTrim, htmlSplit, htmlJoin, htmlReplace.
‘import ”std/c/parsehtml”‘ · ‘use parsehtml‘
- ‘htmlLen(s text) → number‘ - ‘htmlUpper(s text) → text‘ - ‘htmlLower(s text) → text‘ - ‘htmlTrim(s text) → text‘ - ‘htmlSplit(s text, delim text) → list text‘ - ‘htmlJoin(parts list text, delim text) → text‘ - ‘htmlReplace(s text, from text, to text) → text‘

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/parsehtml‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : htmlLen, htmlUpper, htmlLower, htmlTrim, htmlSplit, htmlJoin, htmlReplace.

‘import ”std/c/parsehtml”‘ · ‘use parsehtml‘

```
- `htmlLen(s text) → number`
- `htmlUpper(s text) → text`
- `htmlLower(s text) → text`
- `htmlTrim(s text) → text`
- `htmlSplit(s text, delim text) → list text`
- `htmlJoin(parts list text, delim text) → text`
- `htmlReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parsehtml"
use parsehtml
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés — graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 htmlLen(s text) → number•
 htmlUpper(s text) → text•
 htmlLower(s text) → text•
 htmlTrim(s text) → text•
 htmlSplit(s text, delim text) → list•
 htmlJoin(parts list text, delim text) → text•
 htmlReplace(s text, from text, to text) → text
```

4. Exemples d’utilisation

```
import "std/c/parsehtml"
use parsehtml
```

```
create result = htmlLen(/* args */)
say result
```

```
create result = htmlUpper(/* args */)
say result
```

```
create result = htmlLower(/* args */)
say result
```

```
create result = htmlTrim(/* args */)
say result
```

```
create result = htmlSplit(/* args */)
say result
```

```
create result = htmlJoin(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/parsehtml"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module parsehtml

```
function htmlLen(s text) returns number
end
```

```
function htmlUpper(s text) returns text
end
```

```
function htmlLower(s text) returns text
end
```

```
function htmlTrim(s text) returns text
end
```

```
function htmlSplit(s text, delim text) returns list text
end
```

```
function htmlJoin(parts list text, delim text) returns text
end
```

```
function htmlReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library ‘std/c/parsehtml‘ (tier complex, category complex). Exports 7 function(s): htmlLen, htmlUpper, htmlLower, htmlTrim, htmlSplit, htmlJoin, htmlReplace.

‘import ”std/c/parsehtml”‘ · ‘use parsehtml‘

```
- `htmlLen(s text) → number`
- `htmlUpper(s text) → text`
- `htmlLower(s text) → text`
- `htmlTrim(s text) → text`
- `htmlSplit(s text, delim text) → list text`
- `htmlJoin(parts list text, delim text) → text`
- `htmlReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parsehtml"
use parsehtml
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 htmlLen(s text) → number•
 htmlUpper(s text) → text•
 htmlLower(s text) → text•
 htmlTrim(s text) → text•
 htmlSplit(s text, delim text) → list•
 htmlJoin(parts list text, delim text) → text•
 htmlReplace(s text, from text, to text) → text
```

4. Usage examples

```
import "std/c/parsehtml"
use parsehtml
```

```
create result = htmlLen(/* args */)
say result
```

```
create result = htmlUpper(/* args */)
say result
```

```
create result = htmlLower(/* args */)
say result
```

```
create result = htmlTrim(/* args */)
say result
```

```
create result = htmlSplit(/* args */)
say result
```

```
create result = htmlJoin(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/parsehtml"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module parsehtml

```
function htmlLen(s text) returns number
end
```

```
function htmlUpper(s text) returns text
end
```

```
function htmlLower(s text) returns text
end
```

```
function htmlTrim(s text) returns text
end
```

```
function htmlSplit(s text, delim text) returns list text
end
```

```
function htmlJoin(parts list text, delim text) returns text
end
```

```
function htmlReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/parsehtml"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/parsehtml/

Source (extrait)

```
module parsehtml

function htmlLen(s text) returns number
end

function htmlUpper(s text) returns text
end

function htmlLower(s text) returns text
end

function htmlTrim(s text) returns text
end

function htmlSplit(s text, delim text) returns list text
end

function htmlJoin(parts list text, delim text) returns text
end

function htmlReplace(s text, from text, to text) returns text
end
end
```